

The Round as a Formula

Fair, transparent, real-time multiplayer games with the Tau language: recomputable rounds, statements about the executed lines decided by the engine, and what remains trusted

Second, revised account of the method first described in *Provably Fair Multiplayer Gaming via Formal Verification* (white paper v1.02, December 2025) [18].

Taumorrow · October 2026 · version 2.6 · DOI 10.5281/zenodo.23189301

Abstract

This paper is about games in which several parties move at the same time and one step decides the round: a lottery, a pot shared by its players, a bet against a bank, a hand of cards, a move on a board. Such a round is usually a program run by an operator. The method disclosed here makes the round a formula. The *round function* (inputs, previous state) $\mapsto$ (outputs, next state) is written in the temporal logic of the Tau language, created by Ohad Asor [2] and developed by IDNI AG, and the engine evaluates that text step by step; no separately written program states an outcome of the round in the execution. Properties of the round are put to the same engine as queries on the executed lines, each with its expected verdict and, wherever one exists, a negative control; a round is checked by evaluating it again from its recorded inputs and the state before it; the inputs of every party are bound by commitments before any opening, and what follows from a missing opening is a line of the formula. The method is stated for every game whose round can be written as a block of guarded definitions over the inputs of the round and the previous state; that it carries over beyond the games run is a conjecture (§5.7). No single ingredient is new; what is disclosed is their combination and the kind of evidence aimed at.

What has been run, on one machine, is this. Nine small programs — dice in several constructions, a lottery, a pot that rolls over, the order in which the parties contribute — put 64 queries to the engine, all answered as expected on the public development branch, two of them with an external synthesis tool. A four-player race game was played 16 times to a win, every printed output equal to that of a reference model by the same author; 42 queries on one to five of its executed lines and seven queries on all lines of a whole round, with four and with eight players, were answered as expected, the queries on the four-player round also on the public branch. Single steps were checked from a recorded state. The same rules ran from 16 to 96 blocks of lines; a lottery with up to 64 tickets, a wheel, the smallest form of a card game, and the smallest forms of two of seven newer designs ran with every value equal to a reference computation. A probe put to the engine shows that an equation between amounts kept in bytes is answered as valid where the amounts are not conserved; the method therefore counts a statement about a payout as decided only as an equation together with the bounds that keep every sum inside the byte.

The whole round of the race game was executed only on a build that carries the author's engine changes, which are not merged; there it opens in seconds. On the public branch the run of the same round with 16 blocks did not open under a memory cap of 7.5 GB, while the seven queries on its lines were decided there. One round of the four-player game took a median of 63–81 ms through an in-process host, the evaluation alone.

Designed and not run are the binding of the inputs, the signed records, the marks and the settlement in the lines of the whole round, the network execution, and most classes of games. What remains trusted is the operator, who is the only channel, opens last, sets the marks, and can bias the draw, take moves and abort a game; the engine build; and the agreement of the formula with the rule in words (§7).

1. Introduction

1.1 A round nobody can evaluate again

In a game played over a network by several parties at once, the round is the unit of trust. All players submit at the same time; one computation takes their inputs and the state so far and yields the common random result, the effect of every move, the new state and what each party is owed. Where that computation is a program held by the operator, a player sees its outputs and nothing of the step between. In a scheme described for games of chance over a network the operator commits to a seed by a hash, the player adds a seed, and the player recomputes the random event afterwards [6, §1-§3]; in the texts surveyed, the scheme does not extend to the moves, the state carried between rounds or the settlement.

The method of this paper makes the whole round one object that every party can hold. Every step from the contributions to the outcome is a line of the formula, and the formula that is read is the formula that runs. A round is checked by evaluating the formula again on the recorded inputs and on the state before the round, and by comparing every output. This detects a deviation after the round; it prevents nothing, and it does not show when an input was fixed — that is the task of the binding (§4).

In December 2025 and January 2026 the same author published a white paper [18] and a companion text [19] on writing the rules of such a game as formulas of the Tau language and executing them. This paper takes up the “Validation Roadmap” of [18]: what of it has been run is in §6, and what of the earlier account is corrected is in §3.3. The worked example is a four-player race game with simultaneous moves (§3); it is the large example, not the subject. The subject is the round of any game of the classes of §5.

1.2 What is disclosed, and what is not claimed

In the texts surveyed (§9; the survey was not a search by patent classification) the following combination was not found: a temporal formula in a decidable logic, or formulas with a declared wiring, that states the rule, the carried state, the draw from contributions of the parties and the settlement of a round; that is itself evaluated step by step as the game; and about which the engine that evaluates it is asked the properties, as queries built from the strings it executes. This paper discloses that combination, for every game of the classes of §5, whether or not it has been run here. It claims no single ingredient, no unpredictability of the draw, no compulsion of an operator, no conformity with a regulatory standard, and no priority beyond what is dated (§10).

Five words of the title are delimited here once and used in this sense throughout.

Fair names the aim of such games, not a result of this paper. Used of a round, it means one thing: no party fixes its input knowing another party's input or the draw, as far as the binding of §4 reaches — and that binding is designed.

Transparent means that the text of the executed specification, its hash, the build of the engine and every round record are available to every party, and that every round can be evaluated again from them. It is not a claim about randomness: the formula adds no randomness and hides none.

Provably is used only inside the phrase “provably fair” of the earlier account, which §7 takes apart. In its place this paper puts two phrases and nothing stronger: *recomputable from recorded inputs on a named build*, and *statements about lines of the round, decided by the engine on queries built from the strings it executes*.

Real-time refers to the fact that a round is one step for all parties — nobody waits for another party's move — and to the time that step took in the runs of §6.12. It is not a claim about a network, about the exchange of §4, about a game played by people, or about a bound on the time of a step. A game that is not divided into steps, in which the order of arrival of the inputs decides, is outside the method.

Multiplayer names that several parties contribute to one draw and move in one step. It is carried by the lottery, the pot and the board game as run; the wheel as run has one player contribution and one bank contribution.

1.3 Reading this paper

Sections 2 to 5 state the method; §6 collects what has been run, §7 the limits, §8 what is designed and has not been run. Four words name the evidence: *run*, an engine run with a stored log on a build named by its commit; *computed*, arithmetic by hand, or a small program outside the engine that recomputes the rules on bytes (no engine run); *read*, read in a source text, nothing run; *designed*, described here and neither run nor built.

Four builds of the engine are named throughout (Appendix C); three more were used in §6.10 only (Appendix C). *The named build*, 44b47c42f, is the public development branch at fd536d56e plus the author's engine changes, which are pull requests under review and not merged. *The earlier build*, 5b1842c5e, is an earlier state of the same branch. *The public build*, a739b9025, is the state of the public development branch on 1 October 2026 without those changes; *the later public build*, 30c408493, is a later state of the same public branch.

A *stream* is a sequence of values, one per *time point* $t = 0, 1, \dots$. A *specification* is a formula over input and output streams that the engine evaluates one time point after another, reading inputs and printing outputs; its conjuncts are its *lines*. *valid* ϕ asks whether ϕ , with every variable bound by *all*, holds, and the engine prints T or F; *realizable* asks whether outputs exist for every course of the inputs. Each of these, put to the engine, is a *query*. The *draw* is the common random result of a round; in a game with dice it is the *roll*. A *move* is what a party chooses in a round. A *reference computation* (for the race game: the *reference model*) is an ordinary program by the same author that computes the expected outputs outside the engine. A *mark* is the input *ok_i* by which the operator records whether party *i* took part in the round (§4.1). *As shipped* means the lines of the release of the race game of January 2026.

2. The method

2.1 The round as a function, written as a formula

The object of the method is the round function

$$R : (\text{inputs of round } t, \text{ state after round } t-1) \mapsto (\text{outputs of round } t, \text{ state after round } t)$$

written as a formula of guarded definitions. One game differs from another in four building blocks: the draw (§2.3), the map from the draw to the event of the game, the state transition and the settlement (§2.4). The inputs of a round are the contributions to the draw, the moves and the marks; every state value is an output stream with an initial condition.

One step for all parties. The formula reads the inputs of all parties at one time point and defines every output of that time point. No party's move is an input to another party's move within the round, so no party waits for another, and the round is one evaluation. The same simultaneity makes the binding of §4 necessary: a party that saw the others' inputs before fixing its own would choose with knowledge.

Why the formula is a function. In the specification of the race game every output stream of a time point $t \geq 1$ stands on the left-hand side of exactly one equation, or of every leaf of exactly one tree of conditional equations; right-hand sides and guards read only inputs of t , outputs of $t-1$, and outputs of t defined earlier in a fixed order; every operation is total on bytes. Substituting in that order yields one value for each output, and any assignment satisfying the conjunction must take it. This is read, not run; the engine's recognizer of this shape counted every step of two games without a fallback (§6.6).

What is executed. No separately written program states a roll, a target, a capture, a position or a winner in the execution; the reference model does, as a check. The evaluation is engine code — on the named build partly the author's (§7) — and a query is answered by the engine's decision procedure, a step by its evaluation path.

2.2 Nothing left free, nothing left to the host

Uniqueness per time point does not yet make a run a function of its inputs. The method closes five freedoms. Every state stream has an initial condition $s[0] = c$ with c a constant of the game header; without it the engine fills the first value itself, as it did with 0 in every run of the race game as shipped, and nothing in the text obliges a build to do so. The type of an input excludes values out of range; where the width of a stream does not fit the range, the formula has a line for every value outside it, and no host program filters the input. What a host program would otherwise choose — the winners when several finish in one round, the end of the game, what a party that does not open contributes — is a line. No connection between parts of a network has a default value (§2.6). And the first time point precedes the first input and is not a round; round r is time point r .

2.3 A transparent draw

A *transparent draw* is a common random result computed by lines of the formula from one contribution of every party, the operator among them. The randomness is brought by the hands that contribute; everything that happens to it afterwards is a line that can be read,

run and evaluated again. The term says how the result is computed, not that the contributions were random, independent or fixed in time. For a modulus N the draw is

$$\text{draw}[t] = (d_1[t] + \dots + d_k[t] + (s[t-1] \% N)) \% N, \quad d_i[t] = c_i[t] \% N$$

with one byte per contribution c_i and an optional carried state s , printed like every other output. The sum stays below 256 while $k \cdot (N-1) < 256$; for more summands the sum is reduced after each addition, which stays below 256 for $N \leq 128$ (designed; not run). The sum of contributions modulo N is the construction of [1] and [6] and is not claimed.

Exactness is a count under an assumption. A byte has $256 = 42 \cdot 6 + 4$ values: a single byte taken modulo 6 gives four faces 43 values and two faces 42. If one contributor draws its byte uniformly from 0..251, independently of all other summands, the sum roll is exactly uniform on 1..6; from 0..255 it gives 43 to four faces and 42 to two (computed). The rejection of 252..255 is the honest hand's own act before its commitment; the round never rejects, because a round that rejects gives a redraw to whoever can bring the rejection about. For other moduli the hand stops at the largest multiple of N in a byte: 250 for 10, 222 for 37, 208 for 52, 200 for 100; for every N that divides 256 a byte is exact without any condition (computed). Uniformity is not decided by the engine: it is a statement about all draws at once. The engine decides a symmetry instead — below 255 the next contribution gives the next face, and not without that bound — from which the count follows by hand (§6.2).

Every hand counts, and the last hand chooses. With the other summands fixed, one contribution reaches every outcome: no coalition of the others takes a face from a hidden hand, and a hand that sees the others chooses the face. Which reading holds is decided by the protocol. A contribution that copies $j-1$ others reaches only $N / \gcd(j, N)$ outcomes — a copied contribution turns d into $2d$ — so the commitment of §4.1 carries seat and round. A withheld opening is the same choice with one bit: 0 enters in place of a hand's summand, which can change the outcome.

Several draws, orders, deferred draws. A round may need several draws, each over its own modulus from its own bytes. A draw over the $N!$ orders of N things is the vector of draws over the moduli $N, N-1, \dots, 2$. What no party needs to know before it is revealed is drawn only when it is revealed (a *deferred draw*); each such draw is one more exchange of §4, with its own last look. All three are designed.

2.4 Outcome, state, settlement

The *outcome map* is a total function from the draw and the state to the event of the game: a face, a pocket, the symbols on a line, the rank of a card; a table written as a conditional chain, or arithmetic. The *state transition* is what the next round reads — positions, a point, the cards left, a pot not paid — and the state is what the formula computed, not what somebody stored. The *settlement* gives every seat, and the bank where there is one, the amount returned by the round, with a line for every case in which a party did not take part. Bets are moves and stand inside the commitment of §4.1, so that no bet is placed after the draw is known. A prize shared by several winners is kept in units that divide evenly and written as one case per number of winners, so that no line divides; where a remainder is wanted, a line says where it goes. The method shows the published rule; it does not show that the rule is good for the player.

2.5 Properties as queries on the executed lines; conservation as equation and bound

A property of the round is stated as a query to the engine on the formula that is executed, with its expected verdict and, wherever one exists, a *negative control*: a neighbouring statement that must come out the other way. The query program takes the lines as the strings from which the executed text is joined — one list serves the run and the queries — replaces every stream term `name[t]` by a variable and every `name[t-1]` by a second variable, and asks `valid all ... ((lines) → property)`; for a statement that lines are a function they are taken twice, the second copy renamed. Between the executed text and the asked text lies only this replacement and the quantifier prefix. That a formula expresses the property named is a judgment of the reader.

The kinds of property are: for every draw, on one copy of the lines, a range, a bound on a payout, an invariant through one step (a premise on the previous state, a statement on the new one); on two copies, equal inputs give equal results, two sides never win together, a move that is not read changes nothing; agreement of two ways of writing, such as a payout line against a table; and what a side can force, as a realizable query. *Counting properties* — a return to the player, a frequency, uniformity — are statements about all draws at once; the engine decides them only on a query that repeats the lines once per draw, or they are obtained by a run over all draws or by counting. Not expressible as such queries are uniformity itself, unpredictability, the independence of the contributions, that a mark is true, that a record is issued, and anything about time and delivery.

Conservation over bytes: an equation and a bound. Addition of streams is addition modulo 256, so an equation between sums holds in the arithmetic of the byte and can hold while the amounts it names are not the amounts the rule in words means. Two examples, computed: in the pot that rolls over (\$5.1), two winners of 43 pots would each be printed 2, since $6 \cdot 43 = 258$, and $2 + 2 = 4$ is also $12 \cdot 43 = 516$ in a byte; a proposed split of 250, 18 and 0 satisfies “the three numbers add up to 12” (268 in a byte is 12) while, for six units, a winner would be printed 220 out of a pot of 72. The engine answers such a bare equation as valid; this was put to it (\$6.9). The method therefore takes “decided” for a payout in one sense only: *the equation and the bound*. The bound stands in the query as a premise on the previous state or on each summand (a *fence*), chosen so that the largest sum stays below 256, together with a query that one step keeps the fence; or as terms of the statement itself, one term $(a + b) \geq a$ per addition, since a sum of two bytes that is not smaller than one summand has not wrapped. Every conservation query has a control with a weakened fence, expected not valid, and may be accompanied by the bare equation under the weakened fence, expected valid, which shows that the equation alone carries nothing. That a word-level equation is to be told apart from the equation over whole numbers is known (\$9); what the method adds is that the query is built from the executed lines and that its form is part of the method.

Recomputation is re-evaluation. To check a round is to evaluate the formula again on the recorded inputs and the previous state, on the build named in the game header, and to compare every output — starting from the inputs of the round, not from intermediate values supplied by the program under check. Recomputation detects a deviation after the round; it prevents nothing.

Why a temporal logic, when the domain is finite. Every such property is a statement over finitely many bytes and could be decided by enumeration or by a bitvector solver on a translated copy. The method aims at putting the queries to the object that is run, so that no translation has to be trusted. For hand-written copies this aim is not reached; for the queries on the executed lines it is reached up to the replacement of stream terms by variables.

2.6 Two executions of one round function

One specification. The whole round is one specification with its state in the formula.

A network of part specifications. A program derives part specifications from the lines of the one specification: each line defines one output stream; the defined streams are assigned to parts (the only choice made by hand); what a part reads and does not define becomes an input; a value of the previous time point always becomes an input fed by the wiring with a delay of one step and the initial value of the one specification, so that no part has a look-back. The *wiring table* results by itself: one row per input of a part, naming its one source — an input of the round, a state value of the previous round, a constant of the header, or an output of another part — with delay 0 or 1. The table has no default values; parts and table are hashed together, and the hash stands in the game header. A part that reads only the current time point can be evaluated by any worker process; a state handed in by the wiring makes every part interchangeable. The program that connects the processes (the *messenger*) carries values by the rows of the table and decides nothing. For the race game the derivation gives 10 parts and 70 rows from the 75 lines of §3.1, with a common hash (computed by the derivation program, which starts no engine; Appendix B.3). No network of this kind has been run.

The two executions are meant as executions of one round function; that they coincide is to be shown once per version: hash parts and table together; evaluate the parts from the round inputs and hand the values on by the table; generate the one specification from parts and table by substitution, or compare each cut under its stated assumption; and run the same inputs through both for at least 1 000 rounds. All four are designed. The network is the preferred execution for forms that one specification does not open in the available memory (§6.7); then a statement about the whole becomes queries per part, each for every value of its inputs or under an assumption written as a line, queries that the delivering parts meet the assumption, and rows of the table — and the third is not decided by the engine: the table is the trusted place.

2.7 State kept outside the language

The language has no class of stored data: within a run a line reads the record of the step before, and nothing outlives the engine process. What has to outlive it — a carried pot, a rule in force, a board — lies in a store outside. The seam is the same in every case: the last record goes out, and its state values come back as the initial conditions of the next run. The lines say what follows from every previous state; that the state put in is the state last printed is bound only by the chain of records and the check of one step (§4.4). Sums over many rounds, such as a balance, are formed by whoever holds the records.

3. The worked example: a race game

3.1 The specification

Four players have four pieces each on a track of squares 0..42 (43 is off the track, 0 the entry square, 42 the goal). Every round each player submits a contribution to common dice and the choice of one piece; all chosen pieces move by the same roll; a piece standing still can be captured; a player whose four pieces stand on 42 has won. The whole round is one specification: the sixteen positions and the generator state are output streams with initial conditions. The kinds of line are few: the roll; per player the selected position and the target; per piece a capture flag and the new position; per player a win flag; the count of winners (Appendix B.1).

The lines of the release written as one specification have 18 871 characters, 8 input and 48 output streams, 72 lines. *The method's version as run* has 17 500 characters, 9 input and 49 output streams, 75 lines. It differs in six places: the line `next[0] = 0`; the operator's contribution as a ninth input; the roll as the sum of remainders with the operator's contribution and the carried state; piece choices as 2-bit streams; the capture rule as stated in words; and the count of winners. Both were run (§6.3). The same lines were also written with the record types of the Tau language and run in that form; record types and tables are means of the language developed by IDNI AG and no contribution of this paper.

3.2 The rules chosen

The initial condition of the generator. As shipped the generator state has no initial condition; the engine filled 0 in every run. The method has `next[0] = c0`, a constant of the header; it was run with `c0 = 0`.

The piece choice. In the method it is a 2-bit stream, so a value out of range cannot be written. As shipped it is a byte, and a value above 3 behaves differently in three places of the release (read); 60 rounds with piece choices from 0..255 were run with every value equal to the reference model.

The capture rule. In words, and in the method: a piece that its player did not move, standing on a square other than 0, 42 and 43, is captured if any opponent's target is that square. The formula as shipped requires in addition that the opponent moved the piece with the same number (read); that the two are different functions was decided on handwritten copies (§6.4), and the numbers of captures under the two rules differ several-fold (§6.3).

Settlement. Players who complete their fourth piece in the same round share the prize; the prize is kept in units of 12, so that the share is whole for one to four winners and no line divides. The count of winners has been run; the share line is designed (§4.2).

3.3 The release as shipped, and what is corrected

As shipped, the round is a network of five kinds of part — roll (once per round, reading the previous generator state), targets (4), collision (16), finals (4), winner (4): 29 evaluations per round, each part in a persistent engine process. The positions live in the memory of the host program of the release, the generator state in the engine process; which output is handed to which input is code of that host program, with default values for missing sources; the host program also fixes the range of the piece choice, the winner when sev-

eral have a win flag, and the values for an absent player. Contribution and piece choice arrive together in clear text. The script written per round lists the 29 single evaluations without a link to one another or to the previous state, and it is written by the program that is to be checked (read; the table of its 228 part inputs is in Appendix B.3).

What this paper corrects in the earlier account [18, 19], in short: the shipped dice are not uniform — faces 1 to 4 have 43 preimages of 256 and faces 5 and 6 have 42 — and unpredictability is no property of the formula, since the state is printed and a contribution has eight bits; one contribution reaches every seed, so whoever contributes last and unbound chooses the face; “given initial state $o_{next}[0] = 0$ ” [19] has no line in the formula as shipped; a round is recomputed from its inputs and the state before it, not from a list of single evaluations; and with the inputs bound the operator, without any colluding player, can still bias the draw, take a move by a mark and abort the game (§4.3). The earlier idea that holds is the one this paper builds on: the state is derived, not managed, and a formula cannot deviate from itself — it can still say something other than the rule in words, and that is then the rule that runs.

4. Binding the inputs

A round in which all parties move at once is fair only if no party fixes its input knowing the others’. The binding described here is designed: none of it is built, and none of its lines has been run in the whole round. The protocol is stated for the race game with four players and the operator; §4.5 says what changes with the class.

4.1 Four phases per round

Each game is set up with signature keys generated by each party itself and the hash of the specification. The players have no channel to each other but the operator. Round t has four phases.

1. *Slips*. Each party that accepts the round record $R(t-1)$ (for $t = 1$ the header G) signs a slip $E(i, t)$ with the commitment $C_i[t]$: the hash over tag, game, round, seat, contribution, move and a salt of at least 128 random bits; so does the operator. Seat and round are inside it so that a copied commitment does not open as the copier’s own; the move is inside it so that it is not chosen after the draw is known.
2. *Commitment record*. The operator signs $K(t)$ over the slips that arrived before the slip limit, naming the absent seats, and sends it to the players; it accepts no opening before it has sent $K(t)$, and no party opens before holding it. There is no $K(t)$ without the operator’s slip. The mark of an absent seat is not 1.
3. *Openings*. Before opening, a player checks the operator’s signature on $K(t)$ and in it its own slip, the operator’s slip, a signature by a key of G on every slip, the same predecessor in all slips, and that the absent seats are exactly those without a slip. The players whose slips are in $K(t)$ open contribution, move and salt within the opening limit; the operator checks each opening against its commitment and sets the mark ok_i . A player whose check of $K(t)$ fails does not open; its mark is then not 1.
4. *Evaluation and round record*. The operator opens last, feeds the opened values and the marks to the engine, evaluates one step and issues $R(t)$, which carries the hash of $K(t)$, the marks with their reasons, the openings and all outputs of the step. Before sending its next slip a player checks that this hash is that of the $K(t)$ it received,

checks every opening against its commitment in that $K(t)$, and checks the outputs by the one-step check of §4.4. The slip for $t+1$ carries the hash of $R(t)$ and is thereby its counter-signature.

A round is two exchanges between each player and the operator: slip and commitment record, then opening and round record. Its length is the wait for the slips, the wait for the openings — at most the opening limit — and one evaluation; the first two are set by the header and the network and have not been timed. Whether a slip arrived and whether an opening arrived in time is the operator's word, measured by the operator's clock.

4.2 A withheld opening and the operator's abort, as formula lines

What follows from a missing opening is a rule of the game and a line of the formula (Appendix B.2; designed, not run). A player whose mark is not 1 contributes 0 and does not move in that round; no round is void, the round runs with those who opened. A record in which the operator's own mark is not 1 ends the game without a winner, and the share line returns every player's stake. After the end every state stream keeps its value. No line moves a stake or a share because of a player's mark, and none lets a last remaining player win. The round has an outcome for every value of inputs and marks. Marks as inputs have been run in one small program (§6.2), not in the lines of the whole round.

A mark $ok_i = 1$ can be checked by anyone who holds $K(t)$, from the opening in $R(t)$. A mark $ok_i \neq 1$ cannot: the player can show a signed slip or a valid opening, not that it arrived in time, and the operator cannot show the opposite either.

4.3 What the binding does not give

The operator opens last and knows the draw before anyone else. Without any colluding player it can (a) not open, knowing whether the round would end the game, and so end it with the stakes returned; (b) after seeing every opening, mark any of the four players as not having opened — 16 patterns, of which the 15 that mark somebody replace the marked players' summands by 0, which can change the roll, take their moves and expose their pieces; with all four marked no piece moves, the generator state advances and the round is in effect drawn again, as often as the operator chooses; (c) through k seats of its own, choose among 2^k patterns of opened and withheld. Leaving a slip out of $K(t)$ is the mark of (b) set before any opening; if every slip left in $K(t)$ is the operator's own or a colluding seat's, the operator chooses the draw. With n players there are 2^n patterns; with many seats and a small modulus the subsets reach every outcome in almost every round, and a single mark on a player whose summand is the difference suffices (computed on a model; not run). None of these is a deviation from the specification, and recomputation confirms every such record.

The lines rule out two things: a prize gained by a single such act — apart from the return of the stakes, there is no share without four completed pieces — and a redraw by the operator's own withheld opening, since a round the operator does not open is the last. The operator can also abort by issuing no record at all, which no formula line reaches; against the abort by record stands only that it is signed and attributable. That a party which learns the outcome first can bias it by stopping is the known limit of coin flipping without an honest majority (Cleve [8, p. 364]); here the operator is that party. Variants named and not run, which belong to the protocol and not to the formula: a deposit of the operator

paid to the players on an abort; commitments that anyone can open after a delay [5]; an external beacon as the last contribution; a second channel on which the players compare the hashes of the records.

4.4 The signed chain and the check of one step

The game header G , signed by all parties, holds the hash of the specification (for a network: of parts and table), the build of the engine, the initial value of every state stream, the public keys, the constants and the two limits. Every record carries the hash of its predecessor. An altered input shows as an opening that does not match its signed commitment; a round left out or a state exchanged shows in the chain of hashes (designed; signed records do not exist).

The check of one step. The checker forms the specification of G with every initial condition $s[0] = v$ replaced by the value of s in $R(t-1)$, starts the engine of the build named in G , feeds it the inputs and marks of $R(t)$, and compares the outputs of its first step with those of $R(t)$, stream by stream. Its cost does not depend on the number of earlier rounds. This was run without signatures and without marks (§6.6).

Under six conditions — the named build is public and can be obtained bit-identically; it prints the same values for the same text and inputs on every run and machine; the specification leaves no initial value free; a step can be started from a recorded state; the player's key is the player's own; and the operator's key in G is known to a third party as the operator's from outside the game — a signed round record whose outputs do not follow from the previous signed record and the inputs and marks it states is evidence a third party can check by evaluating one step. It covers nothing else: not a record never issued, not a mark set against the facts, not a slip left out, not an abort, not two records shown to two players who never compare them. An operator that signs two commitment records for one round chooses between two values of its own, and the player who holds the unused one sees this only if it also receives the round record; under the rule for an absent slip the operator can continue two chains, each player named absent in the other's. The record shows what is owed; it does not pay.

4.5 What changes with the class

Lottery and pot. The operator gains nothing by the draw while it holds no ticket and acts with no participant; every lever of §4.3 remains, and a participant acting with the operator brings the interest back. *Bank games.* The operator is the other side of the bet. A line can make its own withheld opening cost it the bet — with $ok_B \neq 1$ the bank pays as if the bet had won; with $ok_P \neq 1$ there is no bet and the stake goes back (Appendix D.2) — but no line reaches a mark set against the facts or a record not issued. *Many contribution bytes or fields per round* are lists in one commitment, in one slip. *Hidden cards* are committed card by card under one root in K , and the checks of hashes and paths are made outside the formula (§5.5). *More than four players:* no byte amount divides by every number of winners from 1 to 8 (the least common multiple is 840), so a larger game needs a line for the remainder or wider amounts. All of this is designed.

5. Classes of games

A class is given by its four building blocks. The table gives each class with its status; the status says no more than was run for it. Appendix D gives the lines.

Class	Exchanges of \$4 per round of play	Status
lottery and pot	one	run: small forms with queries, larger forms without (\$6.2, \$6.8)
bank game with one draw (a wheel)	one	run as an evaluation, without queries, one player contribution (\$6.8)
bank game with state across rounds; machines	one per tick or roll; one	designed
card game with open cards	one; one per card dealt on a decision	smallest form run; larger forms did not start within their limit (\$6.8)
card game with hidden cards	one per round of betting	designed in two forms; a third outside the method
board game with simultaneous moves	one	run, without settlement line and without marks (\$6.3–\$6.7)
game built from a description	—	designed; no generator built
seven designs with a rule, a bank or a bet as a value	one	two smallest forms run; one stage of a third ran without comparison; the rest designed (\$5.8, \$6.11)

5.1 Lottery and pot

In a lottery the operator has no stake in the draw: the draw comes from the participants, the pot goes to the winners. A ticket is a contribution and a pick; a ticket wins if its pick equals the draw; the winners are counted and the pot is shared. In the small form four tickets bet on the faces of dice and the pot is 12 units, so that one to four winners receive 12, 6, 4 or 3 and no line divides; a pick of 0 or above 6 never wins. A pot that is not paid is carried and printed: in the small form a round without a winner carries one pot more, up to a cap of 20, so that at most 21 pots of 12 units are in play and the largest share, 252, fits a byte. In the larger form every ticket pays one unit, the share is the pot divided by the number of winners — a division by a stream — and the remainder is carried.

The cap. At the cap, in the lines as run, the units staked in a round without a winner stand in no line; if they stay with the operator, the operator has a stake in such a round. A line returned that takes what cannot be carried is designed, so that $\text{pot} = \text{winners} \cdot \text{share} + \text{carried} + \text{returned}$; as a query that equation needs the fence on the carried amount and the terms of §2.5. *The limit.* Amounts are bytes; as run, the sum of the reduced contributions has to stay in a byte, so 64 tickets draw one of four numbers; with the sum reduced after each addition the range does not depend on the number of tickets (designed). As run, the draw comes from the tickets alone; the operator’s contribution as a further summand and the marks are designed.

5.2 Bank games and machines

In a bank game a player bets against the operator. With one draw per round — a wheel of 37 pockets, a bet on dice, a threshold on a draw from 0..99 — the player’s and the bank’s contributions are added modulo the number of outcomes, a bet is a move with a kind, a selection and a stake, and the payout is the stake times the factor of the kind, with its own case for an invalid bet. What can be decided for such a game: the pocket is in range; under a bound on the stake, the payout is at most the highest factor times the stake; the pay-

out line agrees with the published table; two opposite bets never win together. The return to the player is a counting property. With state across rounds the same form carries a game in which every tick is a round with its own draw and leaving is a move inside the commitment; it needs one exchange per tick. A machine with reels is a bank game whose outcome map is a set of tables; the return over three reels of 32 stops is a sum over 32 768 stops and is not decided as one query. All but the wheel are designed.

5.3 Card games with open cards

Dealing without replacement is a state: a card is dealt by a draw over the number of cards left, and the deck after the card is an output the next deal reads. Three ways of writing it: counters per rank; a fresh deck per round, digit by digit, each digit with a constant modulus; one stream per position with exchange, one card per time point. The form run takes the index from the generator value modulo the number of cards left, not from a sum draw; its distribution is not exact, and nothing is claimed for it. Games in which a party decides card by card whether another card is dealt are the same form with one exchange per card. Which feature of the run program stopped its larger forms from starting — the stream as a modulus, three deck states chained in one step, or the cascade over running sums — has not been examined (§6.8).

5.4 Board games, moves in turn, games without chance

The inputs are the contributions and the moves of all players; the state is the board and the generator state; the outputs are the roll, the targets, the captures, the new board and the win flags; the settlement is a share of the pot. Moves in turn are the same form with one more state stream naming whose move is read; a move that is not legal is a case of the formula. A game without chance drops the draw and keeps the rule, the state, the settlement and the binding. Only the race game has been run.

5.5 Card games with hidden cards

The formula prints its whole state, so a hidden card is never a stream while it is hidden: it is bound by a commitment outside the formula and becomes an input when it is opened, and the formula recomputes deal, play and settlement from what was opened. Two forms need nothing beyond hash commitments. In the first, the operator commits to an order of the deck, one commitment per position under one root; the players then draw in the open the digits that say which position is dealt to which place; the operator knows every card. In the second, a player commits to a secret share and a common draw is made afterwards in the open; the hidden value is the sum of both modulo N , known only to its holder, who cannot choose it; this carries values that are independent of each other, not hands from one deck. Hands dealt from one deck that no party knows need a protocol of another kind [22] and are outside the method. Both forms are designed.

5.6 Games built from a description

The programs of this paper are each generated, with their queries and a reference computation, by a program of the author. The step beyond is a generator that reads a description of a game as data — draws, tables, outcome cases, state streams with initial values, moves, settlement cases, claims with controls — and writes specification, queries and reference model from that one source, after checking that every output has one definition, nothing is free, every sum has a query that it does not wrap and every conservation query

carries its bound. An oversight in the description stands in all three at once; against it the description carries at least two rounds computed by hand. This is designed; no such generator is built.

5.7 What carries over

That the method carries over from the games run to a class that has not been run is a conjecture. The card game shows that this is not a formality: its larger forms did not start within the limit of the run.

5.8 Seven designs: a rule, a bank or a bet as a value

Seven designs were written to see what becomes playable when a part of the rule is itself a value that the lines read and a query quantifies over. Three figures recur. *A rule or a table set by a player is a value*: the rule in force is a state value, a proposal is an input, a gate line admits it only inside a fence, it takes effect from the next round, and a query whose premise is the fence speaks about every rule that can ever be in force. *A bank or a book is a decided promise*: what a paying party owes, covers and may refuse are lines, and a query decides before acceptance whether they can be kept. *A bet is a formula over the result of the draw*: a seat submits a condition on the drawn number, and “true”, “narrower” and “overlapping” are operations on such values.

Each design has a smallest form and a large form whose characters are counted and whose memory and time are not measured. The answers of their queries were first determined on two models outside the engine, one of them written from the rules in words. Three designs ran in some form (§6.11). *Split the Pot*: three seats stake into a pot on dice; the rule in force splits it between the winners, a jackpot and a return to all, and one seat per round may propose a new split inside stated bounds. *The Open Book*: a seat may be the bank for a bet it writes itself, and the gate takes the book only with a reserve for the worst round; the reserve is booked and never charged, so the answers decide solvency for one round, not for the life of the book — the form with a charged reserve is designed and not computed. *The Bank Keeps Its Word*: two seats bet against a bank whose limit is bounded by promises written as lines that bound an output instead of defining it; whether a bank exists that keeps them against every table is a query on the whole text; stage 0 (definitions and a written rule) and stage 2 (limit left free) were tried. The other four designs — a payout board filled by a seat, a bet as the narrowest true claim, tickets as formulas that may not overlap, and a last word bought at auction — are designed only. What no line of these designs prevents is in §7.

6. What has been run

Everything run is collected here with its build and what it does not show; Appendix A maps each statement to its evidence. The table summarises.

What	Size	Builds	Result
nine small programs, 64 queries (§6.2)	run lines 199 to 2 055 characters	public, named	all as expected on the public build; on the named build the same except program 9
race game, 16 games to a win (§6.3)	17 500 / 18 871 characters	named	every printed output equal to the reference model

What	Size	Buils	Result
42 queries on one to five executed lines (§6.4)	—	public, named	all as expected on both
7 queries on all lines of a whole round, 4 × 4 and 8 × 4 (§6.5)	19 665 / 60 277 characters	named; 4 × 4 also public and later public	T F T T F T T, as expected, on every build that ran them
one step from a recorded state (§6.6)	—	named	eight checks, each equal; an altered output reported
scale, 16 to 96 blocks (§6.7)	up to 240 497 characters	named	20 rounds each, every value equal to the reference
8 blocks; 16 blocks (§6.7)	7 567; 19 665 characters	public	8 blocks ran; 16 blocks did not open under 7.5 GB
lottery, wheel, cards (§6.8)	up to 31 075 characters	named	equal to the reference; two larger card forms did not start
overflow probe (§6.9)	5 closed queries	public, named	T F T T F, as computed beforehand
what a side can force (§6.10)	2 + 4 realizable queries	public, later public, named	public answers as expected by hand; named build differs
two designs, smallest forms (§6.11)	3 515; 7 275 characters	named; one also public	equal to the model; 8 + 8 queries as expected

6.1 Machine, builds, invocation

All runs were made on one machine: Intel N95, 4 cores, 16 GB of memory, Linux Mint 22.3 (kernel 6.8), gcc 13.3. The command-line executables of the named and the earlier build are release builds; the build type of the public builds was not recorded. Runs were made one at a time, each under a memory cap set for it, on a machine that was not idle. The builds print the banner “Tau Language Framework version 0.7.0-alpha (...) by IDNI AG” with their own commit. Times noted by hand for the runs of §6.7 and §6.8 stand in no stored output.

6.2 Nine small programs

Programs 1 to 8 each declare their inputs and outputs, put their queries, then run some rounds; program 9 consists of two realizable queries. A program of the author generates lines, queries and a reference computation from one file; every printed record is compared with the reference. The 64 queries are 5, 8, 6, 8, 10, 6, 9, 10 and 2 per program; most take all lines of one step of the program, once or twice, from the strings that are run, the others are closed statements written for the query. Subjects: many hands and one seed; six faces from a byte; the step of the generator; dice with memory; the sum dice; the operator’s contribution and the marks; a lottery; the pot that rolls over; who moves last.

On the public build all 64 answers were as expected and every printed record equalled the reference computation (programs 1, 3, 6 and 9 as run on 1 October, the other five as extended and run again on 2 October); on the named build the same for programs 1 to 8. Decided, among others: with two hands fixed the third reaches every seed, and with & in place of xor a change in one hand can go unnoticed; below 255 the next contribution gives the next face, and not without the bound; the generator step keeps different values apart; with memory, the new state names a player’s contribution and the roll does not; a zero memory with a zero seed stays zero and rolls 1, and a zero seed does not in general leave

the memory where it was; the sum dice does not depend on the order of the hands, and a copied hand does not reach every face; a hand not marked 1 does not touch the roll; in the lottery, two rounds with equal contributions have the same roll whatever the picks, and equal picks alone do not give it; in the pot that rolls over, under the fence of at most 20 carried pots a round with a winner carries nothing, a round without a winner below 20 carries one pot more (control: always), two, three and four winners share all pots in play (control: three of four), and for a single winner the share divided by 12 is the number of pots — and not without the fence, where 22 pots, 264 units, are printed as 8. Program 9 answered F T on the public build, as expected, and T T on the named build (\$6.10).

What this does not show. The programs are small, and the reference computation is by the same author as the lines. Nothing is bound; contributions, marks and picks are typed in. A query on all lines of a small program is a statement about one step, not about a game over its rounds. The queries on amounts are of the fence form of \$2.5; none carries the terms $(a + b) \geq a$.

6.3 Sixteen games of the race game

A worked run (named build, as shipped). At $t = 1$ the contributions 231, 238, 231, 97 give the seed 143, the new generator state 240 and the roll $240 \% 6 + 1 = 1$; at $t = 2$ the seed 25 gives $240 + 25 = 9 \pmod{256}$, the state 67 and the roll 2; at $t = 3$ the seed 43 gives 110, the state 30 and the roll 1. The printed values agree with this hand computation.

Games to a win (named build, command line). A driver draws random contributions from 0..255 and for each player a random piece not yet at the goal, until a win flag is set; every printed output of every round is compared with the reference model. From one seed six versions were run, from the lines of the release (56 rounds, 2 688 values) through each single change to the method's version (90 rounds, 4 410 values); five further seeds in two versions gave ten games of 56 to 94 rounds and 33 174 values. In every run all values equalled those of the reference model. One version, run also on the earlier build, printed the same 3 184 lines. The method's version in the record form of the language ran one game to a win, 90 rounds, 4 410 values equal. With the capture rule as the only difference, one game had 8 captures under the shipped rule and 38 under the rule in words; ten further games had 8 to 15 and 29 to 48.

What the agreement shows. The reference model is an ordinary program by the same author, sharing constants and stream names with the generator of the specification, not the strings of the lines. The agreement shows that the build evaluates the text as the author reads it, on these games; it is not an independent check of the rule. Piece choices were legal in all but one run, and there was never more than one winner in a round.

6.4 Queries on the executed lines of the race game

The query program of \$2.5 took lines of player 0 and piece 0. All verdicts were as expected: 20 of 20 for the specification as shipped (13 validities, 7 negative controls) and 22 of 22 for the method's version (14 and 8), each on the named and on the public build; the longest took 1.07 s. Decided valid, with controls decided not valid: the roll is in 1..6 (control: at most 5); equal contributions and previous state give the same roll; as shipped, equal new states imply equal contributions of the player, and in the method equal rolls imply equal contributions modulo 6 (controls: the same from equal rolls as shipped; with a copied contribution in the method); the target is at most 42 for every previous position,

choice and roll (control: never 0); the capture flag is 1 only if the piece stood on none of 0, 42, 43, is 0 for the chosen piece, and is 0 or 1 (control: always 0); the finals line keeps positions in 0..43 under its premises, moves the chosen piece to its target, sends a captured piece to 43 and leaves an unchosen uncaptured piece (control: without the premises); the win flag is 1 exactly if the four positions are 42 (control: always 0); the count of winners is at most 4 (control: at most 3). A further 23 queries on hand-written copies were decided as expected on both builds, among them the one non-validity that separates the two capture rules.

What this is not. The largest of these queries takes five distinct lines of the specification; a stream that the lines read and do not define is a free variable; pins and initial conditions are not among the lines taken. A line is queried as a relation at one time point; that this is what the engine makes of it in a run is the interpretation of the logic.

6.5 Queries on all lines of a whole round

The rules of the method's version were generated for P players with K pieces (§6.7) and written as a round over records: one record of inputs per round, one record of outputs. Seven queries take all lines of one step of that round (the initial conditions are not among the lines taken): the roll is a face from 1 to 6 (control: at most 5); if player 0 chooses piece 0, that piece stands on player 0's target after the round, whatever the other players do; a piece standing at 43, 0 or 42 before the round is not captured (control: that piece is never captured); if every piece stood in 0..43 before the round, every piece of player 0 does after it; the count of winners is at most four, and player 0 is marked a winner only with all its pieces on 42. The answers are T F T T F T T, as expected, for 4 players with 4 pieces (19 665 characters) and for 8 players with 4 pieces (60 277 characters) on the named build, each followed by a run of 10 and 5 rounds with every printed record equal to the reference computation; queries and run took about 20 s and 90 s, the larger form peaking at about 1.5 GB. For 4 players the same seven answers came from the public build and from the later public build.

What this does not show. These lines have no marks and no settlement; the queries on players and pieces other than player 0 were not put. The step that carries the fence from one round to every round is the reader's induction, not a verdict. In these generated forms the piece choice is a byte: a choice above $K-1$ moves no piece of the player's own, while its target is still formed and read by the opponents' capture lines (read in the generating program); the inputs drew choices from $0..K-1$. The method's version of §3.1, with a 2-bit choice, does not have this case.

6.6 One step from a recorded state; the recognizer

The specification with its initial conditions replaced by the state recorded after round $t-1$, given the inputs of round t , printed the outputs the continuous run printed at t : 48 of 48 for rounds 1, 2, 17, 40 and 56 of one version, 49 of 49 for rounds 1, 30 and 90 of the method's version — eight checks of about 3 s each, the start-up of the engine, whichever round was checked, while 56 continuous rounds took 11.0 to 11.4 s. In each, a record with an altered roll was reported at exactly that output. The recorded states were engine outputs; signed records do not exist, and the check has not been run on the public build. On the named build the engine's recognizer of the shape of §2.1, in its comparing mode, counted 56 of 56 and 90 of 90 steps of two games with no fallback, mismatch or undecided step.

6.7 Scale, and the public build

The rules of the method's version were generated for P players with K pieces: the roll with $P + 2$ summands, per piece a capture line and a position line (a *block*), $P \cdot K$ blocks per round. Each form ran 20 rounds from one seed on the named build, every output compared with a reference computation.

Players × pieces	Blocks	Characters	Time in all (noted by hand)	Memory	Values equal / different
4 × 4	16	19 665	5.1 s	461 MB	980 / 0
6 × 4	24	37 235	10.4 s	841 MB	1 420 / 0
8 × 4	32	60 277	15.7 s	1 348 MB	1 860 / 0
8 × 6	48	88 185	25.0 s	1 968 MB	2 500 / 0
8 × 8	64	116 093	35.4 s	2 618 MB	3 140 / 0
16 × 4	64	209 469	66.3 s	4 675 MB	3 620 / 0
12 × 8	96	240 497	88.5 s	5 423 MB	4 660 / 0

No run stopped at a limit set for it. Memory grew with the length of the program, 22 to 23 MB per 1 000 characters; the time per 1 000 characters rose from 0.26 s to 0.37 s. No form reached a win in its 20 rounds; captures occurred in every form. These are seven single pairs, not a grid, from one seed, start-up and steps measured only together.

The public build. There, 2 players with 4 pieces (8 blocks, 7 567 characters) printed 540 values in 20 rounds, all equal to the reference computation. The form with 16 blocks — the round of \$6.5 with 4 players, 19 665 characters — did not open: under a memory cap of 3 GB the engine stopped after about 203 s at the cap, on the public and on the later public build alike; under a cap of 7.5 GB and a time limit of 900 s it stopped after 478 s at 7.43 GB when a memory allocation was refused, printing no output. On the named build the same form opens and runs 20 rounds in 5.1 s at 461 MB. The seven queries on its lines were decided on the public build in both attempts (\$6.5). On the public build and this machine, the statements about the whole round were decided, and the run of 16 blocks did not open under a memory cap of 7.5 GB; the named build, which differs from it by the author's engine changes and by an older base, executes it. Which change makes the difference has not been isolated. This is one run per cap, on one machine.

6.8 A lottery, a wheel and a card game

These programs ran on the named build, each against a reference computation of every output of every round, without queries.

Program	Size	Rounds	Time in all (noted by hand)	Memory	Values equal / different
lottery, 4 / 16 / 64 tickets	1 644 / 4 188 / 14 556 characters	12 / 20 / 20	0.3 / 1.2 / 6.7 s	68 / 130 / 462 MB	108 / 420 / 1 380, none different

Program	Size	Rounds	Time in all (noted by hand)	Memory	Values equal / different
wheel, 8 / 32 bets per round	7 915 / 31 075 characters	20 / 20	2.0 / 11.5 s	200 / 710 MB	220 / 700, none different
cards, 2 ranks of 2, 2 players	3 600 characters	8	23.5 s	635 MB	184 / 0
cards, 4 ranks of 2, 2 players; 13 ranks of 4, 4 players	4 874; 12 593 characters	—	did not start within 90 s; 200 s	—	—

With them, division with remainder by a stream, the product of a stake and a factor, and the maximum of the payouts have been run. *What this does not show.* No query was put on these lines; none was run on the public build; none has marks; the lottery has no operator's contribution and no line for what a round without a winner cannot carry beyond the cap. In the lottery with 4 tickets the remainder is never exercised; on the wheel no stake above 7 occurs.

6.9 The overflow probe

Five closed queries over bytes, written for the purpose and not taken from executed lines, were put to the public and the named build. With the split 250, 18, 0 of k units: (1) the three parts add up to $12k$ — valid for every k ; (2) the winner's part is at most $12k$ — not valid. With the split 6, 4, 2 under the fence $k \leq 21$: (3) the equation and the bounds $\text{win} \leq 12k$ and $\text{win} + \text{jack} \leq 12k$ — valid. Without the fence: (4) the equation — valid; (5) the bound on the winner's part — not valid (the model's first counterexample is $k = 22$: 132 against 8). The answers, $\top \text{ F } \top \text{ T } \text{ F}$ on both builds in about 0.2 s, are those computed beforehand on a model. The lesson of §2.5 is thereby run: a bare equation between byte amounts is answered as valid where a winner would be paid more than the pot.

6.10 What a side can force

Program 9 asks, with one bit per hand, whether a hand that fixed its bit one step earlier can meet the other hand's bit again and again (expected: no), and whether a hand that chooses after the other has shown can (expected: yes). With an external synthesis tool on the search path, the public and the later public build answered $\text{F } \top$, the named build $\top \text{ T}$; without the tool no build printed an answer, and the engine reported that realizability is unknown. These queries therefore need that tool.

A small game between two parties over two-bit values was put in four sessions of realizable queries whose answers were worked out by hand: a control pair ($\top \text{ F}$); a chase on a line, in which the pursuer can always catch up again (expected realizable); the same on a ring, where the other party escapes (expected not); and the line without a bound on the other party's moves (expected not). The later public build answered all four as expected, in 0.03 to 1.1 s per query below 65 MB, and reported its decision as a game over the data; a public state of 24 September 2026 and the named build, which starts from an earlier public state (Appendix C), answered differently (the named build: unknown, realizable, realizable for the chase on a line, the ring and the line without a bound). Two local builds that carry the author's engine changes on later public states answered as the later public

build (one run each), so in these runs the difference goes with the older base, not with those changes. For realizable queries the public state is therefore the reference, and every such answer names its build.

6.11 Two of the seven designs, in their smallest forms

The texts and queries were taken mechanically from the design's generator. *Split the Pot* (3 515 characters) ran five rounds on the named build with every record equal to the model — a proposed split of 250, 18 and 0 in round 3 is refused by the gate — and the same records came from the public build. Its eight queries answered T F T F T F T F on both builds, as expected: conservation of every chip *with* the terms that no sum has left the byte, inside the fence, valid; the same under the weaker fence “the three numbers add up to 12”, not valid; the gate keeps the fence (control: the rule never changes); a proposal does not change the payout of its own round; two seats that exchange their inputs exchange their payouts; each with a control. *The Open Book* (7 275 characters) ran five rounds on the named build with every record equal to the model; its eight queries answered T F T F T F T F there, as expected; the reserve is never charged in this form (§5.8). *The Bank Keeps Its Word*, stage 2, with the limit left free and six promises as bounding lines (2 143 characters), printed no step within 120 s on the named build; stage 0, with definitions and a written rule (2 074 characters), ran four rounds in about 60 s, too slow for its size, its values not compared with a model and its queries not put. The large forms and the other four designs have not been run.

6.12 Times

At the command line (named build; start-up and steps). Twenty steps of the race game took 6.25 s on the named and 6.11 s on the earlier build, with identical lines. Truncated to 0, 1 and 56 rounds and run twice each, one version gave a start-up of 2.8 to 3.0 s and 148 and 150 ms per round.

Through an in-process host (named build). At low load, in nine runs, start-up took 2.10–2.35 s, the first round 73.7–114.3 ms, the median of the other rounds 63.3–81.0 ms, the maximum 93–215 ms, peak memory 412–436 MB; while the load rose, the medians were 154–188 ms. In this host time point 0 arrives without a step request; a driver must assign every output to its time point.

The engine's own timers (named build, command line, machine not strictly quiet). Medians per step of the evaluation of the step and of reading its inputs: 0.25–0.88 ms and 0.23–1.98 ms for small programs of 199 to 826 characters; 15.0 ms and 2.7 ms for the round of 4 players (19 665 characters); 36.9 ms and 6.2 ms for 8 players (60 277 characters). The rest of the wall time per round lies outside these timers.

What this says about real time. For the race game with four players, on the named build and this machine, one round was one evaluation of 63–81 ms in the median through the in-process host and about 150 ms at the command line, after a start-up paid once per game. These times concern the evaluation alone: the exchange of \$4 is not built, the network execution has not been run, no game was played by people, and no quiet measurement has been made.

7. Limits

The phrase “provably fair” bundles five statements. (1) *The published rule is the executed rule*: this holds for whoever runs the formula; that an operator ran it would be shown only by the designed records of §4.4. (2) *The rule has the stated properties*: the engine has decided statements about one to five lines of the race game, about all lines of one step of its generated round for 4 and 8 players, and about the lines of the small programs — none about the lines with marks and settlement, none about a game over its rounds. (3) *The outputs follow from the inputs*: run for games to a win against a reference model by the same author. (4) *The inputs were fixed before they were known*: the binding of §4, designed; under it the operator can still bias the draw, take moves by marks, abort, and for a seat acting with it choose the draw. (5) *The randomness is unpredictable and the game is fair in the everyday sense*: this lies outside any formula. For a statement about amounts, “decided” means the equation and the bound (§2.5).

The engine build and the author’s engine changes. It is trusted that the build evaluates the formula as the logic defines and prints the same values on every run and machine; two states of one branch printed equal lines, other machines were not run. The whole round of the race game has been executed on neither of the two public builds tried: on the public build its 16 blocks did not open under 7.5 GB (§6.7). The queries on its lines, the small programs and a form of 8 blocks ran on the public build; a realizable query was answered differently by two builds (§6.10). A result names its build.

The query program. It is trusted that the lines asked are the lines run, with stream terms replaced by variables and a prefix added, and nothing else; this can be checked by comparing the asked texts with the executed text.

The operator. It is trusted that the header names what is run; that a slip “not arrived” and a mark “not opened” are true; that every player sees the same game; that records are issued and reach every player; that payment is made, which nothing enforces; that the other keys are not the operator’s; and that the key called the operator’s is the operator’s, which only a publication outside the game shows. The operator keeps the last look, the marks and with them a redraw, the abort, and — holding a seat — the means to take the others’ moves round after round. In a bank game it is also the other side of the bet. A player can check the header hash and recompute; an omission or a mark set against the facts is visible to the player concerned, who cannot show it to a third party.

The designs of §5.8. A proposal inside the fence is taken without the consent of any other seat, so whoever fills the one place for a proposal in the input record sets the rule — and the operator assembles that record. A seat is an input byte: an offer made in the name of a seat is entered by whoever writes that byte. Where the operator is the paying party and its hand enters the draw, it has both the interest and, opening last, the means. Amounts handed back or left over are named without a recipient in several designs. Each of these is a line still to be written or a matter of the protocol.

Amounts and conservation. The lines keep amounts in one byte. Wider streams have not been run. An equation between amounts in bytes holds modulo 256 (§6.9); the conservation queries of the small programs are of the fence form, queries with the terms $(a + b) \geq a$ ran in one design only (§6.11), and the larger lottery, the wheel and the card game have had no query.

Counting properties are not decided as one query where the space of draws is large; for three reels of 32 stops this is not practical.

Hidden cards. With a deck committed by the operator, the operator knows every card, and so does a seat that acts with it.

Scale and real time. Larger rounds were measured on one machine, 20 rounds each, start-up and steps together, on the named build only. The times per round are those of one game of four players, the evaluation alone; the time of a round over a network is not known.

The specification. It is trusted that it says what the players take the rules to be; what it leaves open the operator may choose, and recomputation confirms it.

Entropy. The 8-bit contributions, the player's own uniform draw and the absence of collusion are trusted.

Standards. The method does not use the means that technical standards for interactive gaming prescribe — independent source review and statistical testing of a cryptographically strong generator [12, §3.2–§3.3]: here the state is public and the randomness comes from the contributions of the parties. Whether such standards apply to a given game is not addressed.

The evidence of this paper comes from one author and one machine, with no independent reproduction, no game played by people, and reference computations by the same author.

8. Designed and not run

The following are described and have not been run: the binding of \$4 — slips, commitment records, openings, signed round records, the chain; the lines with marks, halt, end and share in the whole round; the network execution and the comparison of the two executions; a check of one step on the public build; a game played by people and a replay by a second party; queries with the terms $(a + b) \geq a$ on executed lines of the board game and the lottery; the lottery with returned, with the operator's contribution and with the sum reduced after each addition; bank games with state across rounds, machines, card games beyond the smallest form, hidden cards, moves in turn, games without chance, the generator from a description; several players at one bank table; the large forms of the seven designs, and five of the seven in any form; realizable queries over bytes; a quiet timing run, a second machine, and the time of a round including the exchange of \$4. Next to be run, in this order: the check of one step on the public build; the network execution with open engine processes; the smallest forms of the remaining designs; the scaled forms with start-up and steps measured apart.

9. Related work

Rules as a logic description. GDL-II [20] describes games with simultaneous moves and a random role; “the entailment relation is decidable” [20, p. 998], and the random move is drawn by a controller that “knows all moves” [20, p. 998]. **Synchronous dataflow.** In Lustre the same language writes programs and their properties, and two versions are compared under a stated assumption [15, pp. 1305, 1316]; this is the nearest predecessor

of asking the object that is written. The method differs in what is written — a round with randomness from the parties, their binding and a settlement — and in that the text is evaluated by the engine that answers the queries. **Rules on a ledger.** In ForceMove [9, §2.2] a library on a ledger defines the valid transitions, and payment is enforced; the method has no ledger and enforces no payment. **Replay against registered rules.** US 6,165,072 [10] re-generates a result from the player’s input, a joint seed and “predetermined game rules” [10, claim 1] and states that “the verification software needs to be trusted” [10, col. 18]; WO 01/98860 A2 [3] claims hash commitments by several players, opened and combined [3, claims 1, 3]. **A draw from a sum of contributions.** Andrychowicz et al. [1, §IV-A] and Cen et al. [6, §4] take the sum of contributions modulo the number of outcomes, uniform while one party draws uniformly; the sum draw of §2.3 is that construction and is not claimed. **Cards that no party knows** need protocols such as that of Shamir, Rivest and Adleman [22]; the formula would then keep only betting, pot, rank and settlement over opened cards. **Accountable operators.** PeerReview [13] keeps tamper-evident logs replayed against a reference implementation; Accountable Virtual Machines [14] apply the same to a virtual machine image, name the goal that users can verify “that the provider of the service implements the stated rules faithfully”, and let a segment be replayed from a snapshot. This is the nearest predecessor of §4.4; what differs is that the replayed object is a temporal formula with its state in its outputs, so that the check of a round is the evaluation of one step of that formula. **Fixed-width conservation.** That an equation over machine words differs from one over whole numbers, and that overflow is a proof obligation of its own, is known from the verification of contracts [24]; §2.5 applies it to queries built from the executed lines. **Bets as formulas.** That a bet can be a Boolean formula over events, paying when the formula is true in the outcome, is known from combinatorial betting [25, §3]; the design of §5.8 in which a seat submits a condition on the drawn number takes that object and is not claimed. What §5.8 adds, as a design, is that the condition is a value read by the lines of the round. **Standards.** GLI-19 [12] requires, for instance, that “each possible RNG selection shall be equally likely to be chosen” (§3.2.3) and that a player who does not act in time does not hold up the others (§4.16.3 c).

10. Priority, reproduction

This paper is a disclosure: the method is described so that it is dated and can be followed. Public before it, by the same author: the white paper [18] (first version 13 December 2025, v1.02 15 December 2025); the licence text of its repository in the version of 15 December 2025 and the licence text of the demonstration files (19 December 2025), which name the loop “Commit → Reveal → Validate → Execute” in which the logic validates all moves, as the white paper describes it; a repository of demonstration files (19 December 2025) and later states of it (from 9 July 2026), including a tutorial series on state machines as executed formulas (from 10 July 2026); the companion text [19] (18 January 2026); the reports and pull requests IDNI/tau-lang #153–#179 (September 2026); and a licence text v3.0 in three public repositories of the author (3 October 2026), which describes the method in words. The supplement deposited with this record (DOI 10.5281/zenodo.23189301; version 2.5: 10.5281/zenodo.23163643) holds the specification texts of the race game (§3.1), the queries of §6.4 with their verdicts, the invocation and the games of §6.3. The sources of the in-process host, of the tools and of the operation of a

game, and the logs, are withheld; the method is not. Version 2.6 differs from version 2.5 only in the wording of the supplement (“dice” for “die”); the specification texts, the queries and the verdicts are unchanged.

A reader can reproduce, by hand, the worked run, the arithmetic of the dice and the examples of §2.5; on the public build, the queries of §6.4 as given in the supplement, and those of §6.5 (4 players), §6.9 and §6.10 from their description; on the named build, which is public only as pull requests, the runs of the specifications given in the supplement with their character counts and SHA-256.

Acknowledgements

This paper owes its existence to the work of Ohad Asor: the logic, the result that it is decidable, and the idea that a specification can be executed as it stands are his, realized in the Tau language developed by IDNI AG. The method and its description here are the author’s, who alone is responsible for them.

Appendix A. Statements and their evidence

Evidence is named by kind (§1.3). What the text calls designed has no row. The evidence files are the author’s, local and not public; their numbers refer to the list below the table.

§	Established	Evidence
2.1, 6.6	One value of every output per time point (read); the recognizer’s counts	read (1); run (9)
2.3, 3.3	Arithmetic of the dice; preimages 43/42; exactness under an assumption; copy reaches $N / \gcd(j, N)$	computed (3, 20)
2.5, 6.9	Wrapped amounts (43 pots; 22 pots; the split 250, 18, 0); the overflow probe T F T T F	computed (3, 22); run (23)
6.2	Nine small programs, 64 queries as expected on a739b9025, the same for programs 1–8 on 44b47c42f	run (15, 16, 24)
6.3	Worked run by hand; 16 games, every value equal to the reference model; equal lines on 44b47c42f and 5b1842c5e; record form	computed (3); run (5, 8, 12, 19)
6.4	42 queries on executed lines and 23 on copies, as expected on 44b47c42f and a739b9025	run (7, 10, 12)
6.5	Seven queries on all lines of one step, 4×4 and 8×4 , T F T T F T T; runs equal to the reference	run (25, 26)
6.6	Eight checks of one step; an altered roll reported	run (9)
6.7	Seven scaled forms on 44b47c42f; 8 blocks on a739b9025; 16 blocks not opened under 3 GB and 7.5 GB	run (18, 21, 26)
6.8	Lottery, wheel, smallest card game; two larger card forms did not start	run (17)
6.10	Program 9 with and without the external tool; the four queries of the two-party game on several builds	run (27, 28)
6.11	Two designs, smallest forms, runs equal to the model, 8 + 8 queries; stage 0 and stage 2 of a third	run (29)
6.12	Times at the command line and through the in-process host; the engine’s own timers	run (2, 8, 9, 11, 13, 24, 25)

Evidence files. (1) Program that generates the specification of B.1. (2) Comparison log of 1 October 2026 on a739b9025. (3) Arithmetic by hand, reproducible from the text. (5) Program that generates the round specification, with the reference model and the driver. (7) Query program for the copies. (8) Logs of 1 October 2026: 20 steps on 44b47c42f and 5b1842c5e; games from the first seed. (9) Recognizer counters, command-line times, checks of one step. (10) The 23 queries on copies. (11), (13) Runs through the in-process host under load and at low load. (12) Query program on the executed lines with its texts and outputs; games from further seeds; the run with piece choices from 0..255. (15) Program that generates the nine small programs. (16) Their texts, expected answers and outputs on a739b9025 and 44b47c42f, 1 October 2026. (17) Program and runs of the lottery, the wheel and the card game. (18) Program and runs of the scaled forms. (19) The record form and its game. (20) Counting programs over the 256 values of a byte. (21) Input and output of the form of 8 blocks on a739b9025. (22) Texts and models of the seven designs, and the second computation from the rules in words. (23) The probe of §6.9 with its outputs on both builds, 2 October 2026. (24) Log of the extended small programs on a739b9025 and 44b47c42f, 2 October 2026. (25) Outputs and expected records of the round with 4 and with 8 players on 44b47c42f, 2 October 2026. (26) Logs of the round with 4 players on a739b9025 and 30c408493 under 3 GB, and on a739b9025 under 7.5 GB. (27) Logs of program 9 on three builds with and without the external tool. (28) Sessions, expectations and outputs of the two-party game. (29) Sessions and outputs of the three designs. Files 4, 6 and 14 of the earlier account (sources of the release, the record of the engine changes, the build scripts) are read sources; Appendix C names the release engine and the engine changes.

Appendix B. The race game: lines, networks, records

All streams are `bv[8]` unless stated; `^` is xor, `<<` and `>>` are shifts keeping eight bits, `%` the remainder, `+` addition modulo 256, `g ? A : B` a conditional; 43 is off the track, 0 the entry square, 42 the goal.

B.1 The one specification

As shipped it is run always `(, 72` conjuncts joined by `&&`, and `)`. Stream names for player `p` and piece `k` in `0..3`: `i{p}c`, `i{4+p}pid`, `o0seed`, `o1lfsr`, `o2next`, `o3dice`, `o{4+2p}pos{p}`, `o{5+2p}tgt{p}`, `o{12+4p+k}cap{p}{k}`, `o{28+p}won{p}`, `o{100+4p+k}st{p}{k}`. Order: 16 initial conditions; 8 pins; 4 roll lines; per `p` select and target; 16 capture lines; 16 finals lines; 4 win lines.

```

initial    st_pk[0] = 43                                (no line for next[0])
pin        i[t] & 0xFF = i[t]                            for each of the eight inputs
roll       seed[t] = c_0[t] ^ c_1[t] ^ c_2[t] ^ c_3[t] ; lfsr[t] = next[t-1] + seed[t]
           next[t] = M(lfsr[t]) ; dice[t] = (next[t] % 6) + 1 ; M(x) = (x ^ (x<<3)) ^ ((x ^
(x<<3)) >> 5)
select     pos_p[t] = (pid_p[t] < 1) ? st_p0[t-1] : (pid_p[t] < 2) ? st_p1[t-1]
           : (pid_p[t] < 3) ? st_p2[t-1] : st_p3[t-1]
target     tgt_p[t] = (pos_p[t] = 43) ? 0 : (pos_p[t] + dice[t] > 42) ? 42 : pos_p[t] + dice[t]
capture    cap_pk[t] = (pid_p[t] != k && (H_q1 || H_q2 || H_q3)) ? 1 : 0 , for the opponents q of
p:
           H_q = (pid_q[t] = k && tgt_q[t] = st_pk[t-1]
                 && st_pk[t-1] != 0 && st_pk[t-1] != 42 && st_pk[t-1] != 43)
finals     st_pk[t] = (pid_p[t] = k) ? tgt_p[t] : (cap_pk[t] >= 1) ? 43 : st_pk[t-1]
win        won_p[t] = (st_p0[t] = 42 && st_p1[t] = 42 && st_p2[t] = 42 && st_p3[t] = 42) ? 1 : 0

```

The method's version as run (17 500 characters) has a first conjunct $\text{next}[0] = 0$; a ninth input $i8op$ with its pin ; the roll $((d_0 + d_1 + d_2 + d_3 + d_B + (\text{next}[t-1] \% 6)) \% 6) + 1$ with $d_i = c_i \% 6$, the new state still M of the previous state plus the xor of the five contributions; piece choices $\text{bv}[2]$, compared by $=$; no conjunct on the opponent's piece number in H_q ; and $\text{winners}[t] = \text{won}_0[t] + \text{won}_1[t] + \text{won}_2[t] + \text{won}_3[t]$.

B.2 The lines of the method with marks (designed)

The added inputs are the marks ok_p , ok_B (set by the operator; the test is $= 1$) and c_B . Header constants: c_0 and the unit $u \leq 21$; each stake is $3u$, the prize $12u$.

```

initial    next[0] = c0 ; st_pk[0] = 43 ; end[0] = 0
halt       halt[t] = (end[t-1] = 1 || ok_B[t] != 1) ? 1 : 0
active     act_p[t] = (halt[t] = 0 && ok_p[t] = 1) ? 1 : 0
roll       d_p[t] = (act_p[t] = 1) ? (c_p[t] % 6) : 0 ; d_B[t] = c_B[t] % 6
           roll[t] = (halt[t] = 1) ? 0 : ((Σ_p d_p[t] + d_B[t] + (next[t-1] % 6)) % 6) + 1
state      next[t] = (halt[t] = 1) ? next[t-1]
           : M(next[t-1] + (xor of c_B[t] and of c_p[t] for act_p[t] = 1))
moved      mv_pk[t] = (act_p[t] = 1 && pid_p[t] = k) ? 1 : 0 ; pos_p, tgt_p as in B.1 with
roll[t]
capture    cap_pk[t] = (halt[t] = 0 && mv_pk[t] = 0 && st_pk[t-1] ∉ {0, 42, 43}
           && (OR over q != p : act_q[t] = 1 && tgt_q[t] = st_pk[t-1])) ? 1 : 0
position   st_pk[t] = (halt[t] = 1) ? st_pk[t-1] : (mv_pk[t] = 1) ? tgt_p[t]
           : (cap_pk[t] = 1) ? 43 : st_pk[t-1]
win        won_p[t] = (halt[t] = 0 && st_pk[t] = 42 for k = 0..3) ? 1 : 0 ; winners[t] = Σ_p
won_p[t]
end        end[t] = (end[t-1] = 1 || ok_B[t] != 1 || winners[t] >= 1) ? 1 : 0
share      share_p[t] = (end[t-1] = 1) ? 0 : (ok_B[t] != 1) ? 3u : (won_p[t] = 1) ? 12u /
winners[t] : 0

```

$12u / \text{winners}[t]$ is written as four cases; no line divides, and with $u \leq 21$ no share and no sum of shares leaves the byte; a query that the shares make the prize carries this bound as a premise and the terms of §2.5. Σ , $\notin$ and "OR over" stand for the written-out cases.

B.3 Two networks

Derived (designed; derived by a program that starts no engine). From the 75 lines of the method's version: the parts die (the four roll lines), move_p (select and target of player p), state_p (capture, position and win lines of player p) and count, 10 parts; 70 rows of wiring, 33 of them with delay 1 and 13 from outside; no input has a default value. Every row names the reading part, the input, the source (a part or "external"), the stream, the delay and, for delay 1, the initial value. The common SHA-256 over the order of the parts, the SHA-256 of each part and the table is 061f1cbd5324411612ce8940c3a3685bad01d36341891ad28868d4cf3c53f95f.

As shipped (read, not run). One row per input name; in brackets the default of the host program of the release:

```
roll          i{j}c <- c_j [0]
targets_p     i0dice <- roll.o3dice ; ilpid <- pid_p [0] ; i{2+j}pc{j} <- st_pj [43]
collision_{p,k} i0pid <- pid_p [0] ; ilpcid <- constant k ; i2pcpos <- st_pk [43]
               pair j = 1..3: i{1+2j}otherpid <- pid_qj [0] ; i{2+2j}othert <-
targets_qj.o1target [43]
finals_p      i0pt <- targets_p.o1target [43] ; ilpid <- pid_p [0]
               i{2+j}pc{j}cap <- collision_{p,j}.o0cap [0] ; i{6+j}pc{j}old <- st_pj [43]
winner_p      i{j}pc{j} <- finals_p.o{j}pc{j} [43]
next state    st_pk <- finals_p.o{k}pc{k} [keeps the old value]
generator state held in the engine process of the game
```

$4 + 24 + 144 + 40 + 16 = 228$ part inputs. The finals part moves piece 3 for every choice $\text{pid} \geq 3$, where the one specification moves no piece for a choice above 3; on hand-written copies the two agree under $\text{pid} < 4$ (§6.4).

B.4 Record formats and canonical encoding (designed)

Every record is a JSON object in the canonical encoding of RFC 8785 [21]; H is SHA-256 over those bytes; signatures are Ed25519 over the same bytes. Any encoding that gives one byte sequence per content, any collision-resistant hash and any public-key signature scheme serve as well.

```

spec      one specification: H(specification file)
          network: H({parts: [H(part file), in the derived order], table: H(table)})
G         {tag: "round/header/1", spec, build: {commit, binary: H(binary), options},
          initial: [value of every state stream], keys: [public keys, seats 0..n], sliplimit,
limit, unit}
          game = H(G) ; signed by all
C_i[t]    H({tag: "round/commit/1", game, round: t, seat: i, contribution, move, salt})      salt ≥
128 random bits
E(i,t)    {tag: "round/slip/1", game, round, prev: h(t-1), seat, commit: C_i[t]}          signed
by i
K(t)      {tag: "round/commitments/1", game, round, prev, slips: [the signed slips that arrived],
absent: [seats]}
                                                signed
by the operator
R(t)      {tag: "round/record/1", game, round, prev, commitments: H(K(t) with its signature),
          seats: [per seat: mark, reason, and where the mark is 1: contribution, move, salt],
          outputs: [values at t]}
                                                signed
by the operator
chain     h(0) = H(G with its signatures) ; h(t) = H(R(t) with its signature)
one step  the specification of G with each `s[0] = v` replaced by `s[0] =` the value of s in R(t-
1);
          its first step, on the inputs and marks of R(t), must print the outputs of R(t)

```

For a seat whose mark is not 1 the record carries no contribution and no move; the step is evaluated with both 0. The opening window closes when `limit` has elapsed after `K(t)` was sent; `sliplimit` is the time within which the slips are to arrive; both are measured by the operator's clock. Several contribution bytes or a move of several fields are lists under one commitment.

Appendix C. Builds and invocation

Builds. The named build 44b47c42f carries the author's engine changes: IDNI/tau-lang #154-#173 (13 pull requests with their reports) and #174-#179 (pull requests #175, #177, #179, which carry the recognizer), five later commits on the branch of #179, and IDNI/parser #21; it dates from 30 September 2026 and starts from fd536d56e. The earlier build 5b1842c5e lacks the last three commits. The public build a739b9025 is the public development branch on 1 October 2026; the later public build 30c408493 is that branch on the evening of the same day; a public state 36e65f765 of 24 September 2026 was used in §6.10 only. Two local builds that set the author's engine changes on a later public base (3f5f1bf84 on a739b9025, and ae7f8d16c on 30c408493) were used in §6.10 only. The engine of the release of the race game is 16063155 (December 2025), read in source only.

Invocation. The race game: `tau --block-max-splits 1 -b false < file` on 44b47c42f and 5b1842c5e, and for the queries on a739b9025; the start-up of the whole round on a739b9025 ran without options. The small programs: `tau -q --block-max-splits 1 < file` under a memory cap of 3 GB. The scaled forms: `tau --block-max-splits 1 -b false < file` on 44b47c42f. The round of §6.5 on the public builds: under caps of 3 GB (both public

builds) and 7.5 GB with a time limit of 900 s (a739b9025). The 20-step input of §6.3 repeats three rounds: (231, 238, 231, 97; 1, 3, 1, 0), (228, 155, 72, 46; 0, 3, 3, 1), (7, 32, 30, 18; 1, 1, 0, 3).

Appendix D. Classes of games: the lines

Notation as in B.2: schematic, over plain streams; c_i a contribution, op the operator's contribution. Lines marked *run* were run in the record form of the language; all others are designed and stand in no generated text.

D.1 Lottery and pot

Run (§6.2, programs 7 and 8): four tickets on the faces of dice.

```
roll      roll[t] = (((c_1[t] % 6) + (c_2[t] % 6) + (c_3[t] % 6) + (c_4[t] % 6)) % 6) + 1
win       w_i[t] = (pick_i[t] = roll[t]) ? 1 : 0 ; winners[t] = w_1[t] + w_2[t] + w_3[t] + w_4[t]
share     share[t] = (winners[t] = 1) ? 12 : (winners[t] = 2) ? 6 : (winners[t] = 3) ? 4 :
(winners[t] = 4) ? 3 : 0
pots      pots[t] = carried[t-1] + 1 ; carried[0] = 0
(program 8)
share     as above with 12·pots, 6·pots, 4·pots, 3·pots, written with shifts and additions
carried   carried[t] = (winners[t] != 0) ? 0 : (pots[t] >= 20) ? 20 : pots[t]
```

Run (§6.8): N tickets, numbers $0..m-1$, one unit per ticket; $m = 16$ for 4 and 16 tickets, $m = 4$ for 64; $cap = 255 - N$.

```
number    number[t] = ((c_0[t] % m) + ... + (c_{N-1}[t] % m)) % m
win       w_i[t] = (pick_i[t] = number[t]) ? 1 : 0 ; winners[t] = w_0[t] + ... + w_{N-1}[t]
pot       pot[t] = carried[t-1] + N ; carried[0] = 0
share     share[t] = (winners[t] = 0) ? 0 : pot[t] / winners[t]
carried   carried[t] = (winners[t] != 0) ? pot[t] % winners[t] : (pot[t] > cap) ? cap : pot[t]
```

Designed: $returned[t] = (winners[t] = 0 \ \&\& \ pot[t] > cap) ? pot[t] + (256 - cap) : 0$, so that $pot = winners \cdot share + carried + returned$; the number reduced after each addition; the operator's contribution as a summand; the marks. Division with remainder cannot wrap; the addition $carried[t-1] + N$ stays in the byte under $carried \leq cap$, which the carried line keeps from the initial 0 and which no query has asked.

D.2 Bank games

Run (§6.8), without queries: a wheel with 37 pockets and N bets per round.

```
pocket    pocket[t] = ((player[t] % 37) + (bank[t] % 37)) % 37
pay       pay_b[t] = (stake_b[t] > 7) ? 0
           : (kind_b[t] = 0) ? ((pocket[t] = num_b[t]) ? stake_b[t] * 36 : 0)
           : (kind_b[t] = 1) ? ((pocket[t] != 0 &\& pocket[t] % 2 = 0) ? stake_b[t] * 2 : 0)
           : (kind_b[t] = 2) ? ((pocket[t] >= 1 &\& pocket[t] <= 18) ? stake_b[t] * 2 : 0)
           : (kind_b[t] = 3) ? ((pocket[t] >= 1 &\& pocket[t] <= 12) ? stake_b[t] * 3 : 0) :
0
staked    staked[t] = stake_0[t] + ... + stake_{N-1}[t] ; top[t] = max(pay_0[t], ..., pay_{N-1}[t])
```

The largest payout, 7 units at 36 times, is 252. staked carries no bound: with free bytes it leaves the byte. *Designed:* several players at one table, up to seven summands without reduction; bets on a set of pockets; under the binding

```
pay      pay[t] = (ok_B[t] != 1) ? the payout as if the bet had won : (ok_P[t] != 1) ? stake[t] :
the payout line
```

and a game in which every tick is a round (k the level, h(k) the chance of a stop, mult(k) the factor):

```
draw      x[t] = ((c_p[t] % 100) + (op[t] % 100)) % 100 ; cr[t] = (x[t] < h(k[t-1])) ? 1 : 0
in        in_p[t] = (k[t-1] = 0) ? ((bet_p[t] != 0) ? 1 : 0) : (cr[t] = 1 || out_p[t] = 1) ? 0 :
in_p[t-1]
held      held_p[t] = (k[t-1] = 0) ? bet_p[t] : held_p[t-1]
leave     pay_p[t] = (in_p[t-1] = 1 && out_p[t] = 1) ? held_p[t-1] × mult(k[t-1]) : 0
level     k[t] = (cr[t] = 1 || k[t-1] = kmax) ? 0 : k[t-1] + 1
```

A machine: per reel stop_r[t] = ((c_r[t] % 32) + (op_r[t] % 32)) % 32, the symbol a chain over the 32 stops, the window the same chain at the neighbouring stops, the payout a chain over the winning combinations.

D.3 Card games

Run in the smallest form (§6.8): a pot on two cards from one deck of R ranks with C cards each; state: one counter per rank, the number of cards, the generator value mem and the carried pot; pre, mid, deck are three states of the deck in one step.

```
values     x1[t] = M(mem[t-1] + (c_0[t] ^ ... ^ c_{N-1}[t])) ; x2[t] = M(x1[t]) ; mem[t] = x2[t]
shuffle    pre_r[t] = (cards[t-1] < 2) ? C : deck_r[t-1] ; precards[t] = (cards[t-1] < 2) ? R·C :
cards[t-1]
index      il[t] = x1[t] % precards[t]
rank       left[t] = (il[t] < pre_0[t]) ? 0 : (il[t] < pre_0[t] + pre_1[t]) ? 1 : ... : R-1
take       mid_r[t] = (left[t] = r) ? pre_r[t] + 255 : pre_r[t] ; midcards[t] = precards[t] + 255
second     i2[t], right[t], deck_r[t], cards[t] : the same from mid, with x2[t]
result     result[t] = (left[t] > right[t]) ? 0 : (right[t] > left[t]) ? 1 : 2
pot        as in D.1 (larger form)
```

Designed: the index as a sum draw with one case per number of cards left and a constant modulus in each case; a fresh deck per round, digit s by $r_s[t] = ((c_s[t] \% (N-s)) + (op_s[t] \% (N-s))) \% (N-s)$; one stream per position with exchange, $d_k[t] = (r[t] = k) ? last[t] : d_k[t-1]$. For hidden cards, first form: the operator's commitments $L_j = H(tag, game, hand, j, card_j, salt_j)$ under one root, the open digits q_s over the moduli $52 - s$, and betting lines put_p, in_p, pot, tocall over public inputs; second form: $h_p[t] = (open[t] = 1) ? (((x_p[t] \% N) + (y_p[t-1] \% N)) \% N) : 0$. The pot $put_0 + \dots + put_3$ needs bounds before a statement about it carries units.

D.4 The schema of a generator (designed)

```
game = { draws:    [ {name, modulus N, parties}, ... ],      tables: { name: [constants] },
         outcome: [ {out, cases: [(guard, value), ..., (else, value)] } ],
         state:   [ {name, init, cases: [...]} ],            moves:   [ {name, width or range} ],
         settle:  [ {seat, cases: [...]} ],                  claims:  [ {lines, property, expect,
control} ] }
```

References

Entries [1], [3], [6], [8]–[10], [12]–[15], [20] and [22] were read in the primary text; [5], [21], [24] and [25] were not and are named as reported. A year with † is taken from the address the text was retrieved from.

[1] Andrychowicz, Dziembowski, Malinowski, Mazurek. “Secure Multiparty Computations on Bitcoin.” University of Warsaw, †2013.

[2] Ohad Asor. “Guarded Successor: A Novel Temporal Logic.” arXiv:2407.06214, 2024.

[3] Barber. “Method Providing for a Verifiable Game-of-Chance Played Even over a Computer Network.” WO 01/98860 A2, international filing date 22 June 2000.

[5] Boneh, Naor. “Timed Commitments.” 2000.

[6] Cen, Fang, Jaba. “Provable Fairness.” †2019.

[8] Cleve. “Limits on the Security of Coin Flips When Half the Processors are Faulty (Extended Abstract).” ACM, 1986, pp. 364–369.

[9] Close, Stewart. “ForceMove: an n-party state channel protocol.” Magmo Research, 2018.

[10] Davis, Campbell. “Apparatus and Process for Verifying Honest Gaming Transactions over a Communications Network.” US 6,165,072, granted 2000.

[12] Gaming Laboratories International. “GLI-19: Standards for Interactive Gaming Systems.” Version 3.0, 2020.

[13] Haeberlen, Kuznetsov, Druschel. “PeerReview: Practical Accountability for Distributed Systems.” SOSP 2007, ACM.

[14] Haeberlen, Aditya, Rodrigues, Druschel. “Accountable Virtual Machines.” †2010.

[15] Halbwachs, Caspi, Raymond, Pilaud. “The Synchronous Data Flow Programming Language LUSTRE.” *Proceedings of the IEEE* 79(9):1305–1320, 1991.

[18] Taumorrow. “Provably Fair Multiplayer Gaming via Formal Verification.” White paper v1.02. Repository taumorrow/provably_fair_gaming, file PROVABLY_FAIR_GAMING_WHITEPAPER.MD; first version commit 6c0683c (13 December 2025), v1.02 commit 1829141 (15 December 2025).

[19] Taumorrow. Companion text to [18], in a public repository of the author; first version commit bc99f63 (18 January 2026), cited state commit 61ff19e (18 January 2026).

[20] Thielscher. “A General Game Description Language for Incomplete Information Games.” *Proc. AAAI-10*, 2010, pp. 994–999.

[21] “JSON Canonicalization Scheme (JCS).” RFC 8785. Named as a scheme.

[22] Shamir, Rivest, Adleman. “Mental Poker.” In: *The Mathematical Gardner* (ed. Klarner), 1981, pp. 37–43.

[24] Hajdu, Jovanovic. “solc-verify.” arXiv:1907.04262, 2019.

[25] Chen, Fortnow, Nikolova, Pennock. “Combinatorial Betting.” *SIGecom Exchanges* 6(3), 2007.

Numbers are kept from the earlier account; entries no longer cited are omitted. SHA-256 and Ed25519 are named as schemes.

© 2026 Taumorrow. This text may be shared verbatim with attribution for noncommercial purposes (CC BY-NC-ND 4.0, <https://creativecommons.org/licenses/by-nc-nd/4.0/>); the licence concerns the text, not the method. The Tau language is © IDNI AG under its own licence.